

Agente evaluador

Guía de puesta a punto del grupo

Parcial · Programación de y con Agentes de IA · MBA UCEMA · 2026 2T

Prof. Alfredo B. Roisenzvit

Entrega del parcial	Jueves 10/9, 18:59 — link al repo en la actividad <i>Parcial</i> del campus
Prueba de fuego	Jueves 10/9, en vivo, esa misma noche
Trabajo final (individual)	Domingo 13/9, 23:59 — lo corrige el evaluador que gane el jueves

Este documento es para **imprimir**. Las secciones 1 a 3 las lee todo el grupo; de la 4 en adelante cada uno trabaja sobre **su** ficha. Los cuadraditos son para tachar a mano.

Regla de la casa que atraviesa todo esto

Nadie del grupo escribe código: se construye **describiendo, iterando y documentando**, con la IA como tutor. Si algo no funciona, la pregunta no es si la IA puede — es qué contexto, qué herramienta o qué instrucción le falta.

1 · Qué hay que entregar

Un repositorio público de GitHub por grupo, con **cuatro piezas** y una estructura de nombres que no se negocia: la corrección la asiste un agente, y un agente corrige lo que puede leer.

Las cuatro piezas

Pieza	Qué es	Peso
La rúbrica ejecutable	La rúbrica oficial del trabajo final vuelta aplicable por una máquina: escalas por nivel, qué evidencia exige cada puntaje, ejemplos de nivel alto y bajo.	25
El agente corrector	El system prompt más las herramientas que necesite. Recibe un repositorio y devuelve puntaje por dimensión, justificación citando evidencia, y una sugerencia de mejora. Formato idéntico en cada corrida.	25
Tres casos de prueba	Uno excelente, uno flojo y uno tramposo. El agente tiene que puntuar alto al primero, bajo al segundo y detectar al tercero.	20
La calibración	La evidencia de que las notas del agente coinciden con el criterio humano del grupo: dónde no coincidían, qué ajustaron, cómo quedó después.	15
Proceso grupal	Historia de commits, iteraciones de la rúbrica, decisiones registradas. No es una pieza: es cómo trabajan.	15

La estructura obligatoria del repositorio

README.md	el README estandar de la materia + integrantes
rubrica.md	la rubrica ejecutable
agente/	system prompt y configuracion del corrector
casos/excelente/	caso de prueba 1
casos/flojo/	caso de prueba 2
casos/tramposo/	caso de prueba 3
calibracion.md	la evidencia de calibracion

Esos siete nombres tienen que estar tal cual. Lo demás (*corridas*/, *DECISIONES.md*, este PDF) se agrega libremente y suma.

Y además: la prueba de fuego

El jueves a la noche, cada evaluador corrige **en vivo casos que nunca vio**, delante de la clase. De ahí sale el agente evaluador de la materia: uno solo, el que va a corregir los trabajos finales de todos. No alcanza con que el repo esté lindo — el agente tiene que **correr** el jueves, sobre un repositorio ajeno, sin que nadie lo acomode antes.

2 · Puesta a punto del entorno

Los cuatro, el primer día. No lleva más de una hora y se hace una sola vez.

2.1 Cuenta de GitHub — los cuatro

- [] Creá tu cuenta en **github.com** si no tenés (es gratis).
- [] Anotá tu **nombre de usuario**: lo vas a necesitar para que te inviten al repo.

2.2 GitHub Desktop — los cuatro

Es la aplicación gráfica oficial para usar Git sin terminal. Es la vía recomendada de la materia para no programadores.

- [] Descargá e instalá **GitHub Desktop** (desktop.github.com).
- [] Abrilo e iniciá sesión con tu cuenta.

Si algo falla

No busques un instructivo: pegáselo a tu tutor IA con el error textual. Ese ida y vuelta **es** el contenido de la materia, no un desvío.

2.3 El repositorio del grupo — lo hace UNO solo

- [] Creá un repositorio **público** en GitHub. Nombre sugerido: **agente-evaluador**.
- [] Marcá "Add a README file" para que el repo nazca con algo adentro.
- [] Settings → Collaborators → invitá a los otros tres por su usuario.
- [] Pasá el link por el grupo. Los otros tres aceptan la invitación por mail.

2.4 Bajarse el repo — los cuatro

- [] En GitHub Desktop: **File** → **Clone repository** y elegí el repo del grupo.
- [] Elegí una carpeta local que te acuerdes. Esa carpeta **es** el repositorio.

2.5 Cargar los archivos base — lo hace UNO solo

El esqueleto ya está armado: rúbrica, agente, los tres casos, calibración y las corridas de referencia. Hay que copiarlo dentro de la carpeta clonada.

- [] Copí todo el contenido de la carpeta **agente-evaluador** dentro de la carpeta clonada.
- [] En GitHub Desktop vas a ver la lista de archivos nuevos. Escribí el mensaje de commit y hacé **Commit to main** y después **Push origin**.

Mensaje sugerido:

```
base: esqueleto del evaluador (rubrica v1.2, agente, 3 casos, calibracion)
```

- [] Los otros tres: en GitHub Desktop, botón **Fetch origin / Pull origin** para traerse los archivos.

2.6 Tu primer commit — los cuatro, el mismo día

Abrió **README.md** con cualquier editor de texto, completá **tu fila** de la tabla de integrantes (nombre, usuario de GitHub, carril) y commiteá **sólo esa línea**.

Mensaje: `readme: agrego mis datos y mi carril`

Por qué esto importa más de lo que parece

Son cuatro commits, de cuatro autores distintos, el primer día. Es la forma más barata de que el *git log* muestre desde el arranque que el grupo existe — y el proceso grupal pesa **15 puntos**.

3 · Los cuatro carriles

Cuatro líneas de trabajo que casi no se pisan, para poder avanzar en paralelo sin conflictos de archivos.

Carril	Quién	Archivos que toca	Qué implica
A Rúbrica		rubrica.md	La pieza de mayor peso. Cada requisito tiene que poder responderse sí/no citando evidencia. Si un requisito necesita interpretación, está mal escrito.
B Agente		agente/*	El contrato, el esquema de salida, la plantilla y las banderas. Dueño también del determinismo: si dos corridas difieren, es de este carril.
C Casos		casos/*	Los tres mini-repos y sus ESPERADO.md. El más divertido y el más importante: sin un tramposo difícil, el evaluador no se prueba. Un caso que el sistema aprueba siempre no mide nada .
D Calibración		calibracion.md corridas/	Corre el agente, compara contra ESPERADO.md, documenta los desacuerdos. No cambia la rúbrica: la discute . Quien la cambia es el carril A.

Regla de conflicto

Nadie edita el archivo de otro carril. Si el carril D encuentra un problema en la rúbrica, lo **comenta** en *calibracion.md*; lo **cambia** el carril A. Así el *git log* muestra quién decidió qué, que es exactamente lo que se evalúa.

Calendario sugerido

Día	Quién	Qué
1	Los cuatro	Puesta a punto (sección 2) + tu fila del README + tu primer commit.
1	A / B / C	Cada uno lee sus archivos de punta a punta y hace una mejora propia.
2	Los cuatro	Calibración a ciegas (sección 6). Cada uno puntúa solo, sin hablar con los demás.
2	D → A	D junta las cuatro columnas y marca desacuerdos. A ajusta la rúbrica a v1.3.
3	B	Prueba de estabilidad: tres corridas de cada caso, sesiones limpias, puntaje idéntico.
3	C o D	Correr el agente sobre un repo real y ajeno . La prueba que más se parece a la del jueves.
4	B	Completar el cuadro de costos de <i>agente/config.md</i> con precios consultados y fechados .
4	D	Cerrar <i>calibracion.md</i> §5 con las tablas llenas.
4	Los cuatro	Ensayo de la prueba de fuego (sección 7) y checklist de entrega (sección 8).

4 · Tu ficha

Cuatro fichas, una por carril. Buscá la tuya, tachá lo que vayas haciendo, y commiteá cada ítem por separado con el mensaje sugerido.

CARRIL A — Rúbrica ejecutable

`rubrica.md`

Sos el dueño del criterio. Nadie más toca este archivo. Cuando el carril D encuentra un desacuerdo, la decisión de qué cambiar es tuya, y se registra en la tabla de versiones del final del archivo.

Qué tenés que completar

- [] Leé **rubrica.md** entera, de punta a punta. Son cinco dimensiones con cuatro requisitos cada una: veinte preguntas de sí/no.
- [] Para cada uno de los 20 requisitos preguntate: **¿puedo responder sí o no citando un archivo?** Los que necesiten interpretación, reescribilos.
- [] Revisá que cada dimensión tenga sus dos **anclas** (ejemplo de nivel 4 y de nivel 1) y que sean concretas, no adjetivos.
- [] Después de la calibración a ciegas: aplicá los cambios que salieron, subí la versión a **1.3** y completá la fila en la tabla de versiones diciendo **qué cambió y por qué**.
- [] Chequeo final: los pesos siguen siendo **30 / 25 / 15 / 15 / 15**. No se tocan nunca.

Mensajes de commit — así, no "cambios"

```
rubrica: reescribo R2.4 y R5.2, no se podian responder sin interpretar
rubrica v1.3: anclo R2.1 tras el desacuerdo D-11
```

CARRIL B — El agente corrector

agente/*

Sos el dueño del contrato y del **determinismo**. Si dos corridas del mismo repositorio dan resultados distintos, el problema es tuyo, y no se arregla bajando la temperatura: se arregla escribiendo una regla que diga qué hacer con la duda.

Qué tenés que completar

- [] Leé **agente/system_prompt.md** entera y corréla una vez sobre *casos/excelente/*. Compará tu salida contra *corridas/*.
- [] Verificá que la salida valide contra **agente/esquema_salida.json**: mismos campos, mismo orden, ninguno omitido.
- [] **Prueba de estabilidad**: tres corridas de cada caso en sesiones limpias. Se exige puntaje **idéntico**, no parecido. Guardá el resultado.
- [] Completá el cuadro de costos de **agente/config.md** §4 con precios **consultados el día de la entrega, con fecha y fuente**. Está vacío a propósito: publicar un costo sin fuente es la bandera G7 de nuestra propia rúbrica.
- [] Probá los dos caminos de ejecución (repo clonado y repo en .zip) y anotá qué se pierde en el segundo.
- [] **Verificá el aislamiento de directorios** (*agente/config.md* §3): el repo evaluado se clona **fuera** del directorio de trabajo del evaluador. Y confirmá que el informe trae **config_agentes_hallada** lleno cuando el repo tiene un *AGENTS.md*. Si viene vacío, el agente no lo buscó.

Mensajes de commit — así, no "cambios"

```
agente: primera corrida propia, guardo la salida cruda
config: precios consultados el 9/9, costo por corrida calculado
```

CARRIL C — Los casos de prueba

casos/*

Sos el dueño de la dificultad. Un caso de prueba que el evaluador aprueba siempre **no mide nada**: puede estar midiendo la calidad del evaluador o la facilidad del caso, y no hay forma de saber cuál. Tu trabajo es que no sea lo segundo.

Qué tenés que completar

- [] Leé los tres casos y sus **ESPERADO.md**. Podés reconstruir de memoria qué debería encontrar el evaluador en cada uno.
- [] El tramposo va por su **versión 2** y tiene **seis** vectores de ataque. Leé *casos/tramposo/ESPERADO.md* para saber dónde está cada uno y por qué el sexto (un *AGENTS.md*) es de una clase distinta a los otros cinco.
- [] **Sumá un séptimo vector**, y que sea de una clase todavía no cubierta. Pista: los seis actuales están en archivos de documentación o de configuración. ¿Qué pasa si la instrucción está dentro de la **salida de una corrida**, en un campo que el evaluador *tiene* que leer para hacer su trabajo? Agregalo a la tabla del *ESPERADO.md* con su efecto esperado.
- [] Verificá que *excelente* y *flojo* siguen **sin banderas**: un evaluador paranoico que castiga la honestidad es tan malo como uno que se deja engañar.
- [] Rehacé a mano las cuentas de los tres casos. Si un número de un *ESPERADO.md* no cierra, el error es nuestro, no del agente — ya nos pasó una vez.

Mensajes de commit — así, no "cambios"

```
casos: sumo un sexto vector de inyeccion en un archivo fuera del README
casos: corrijo la aritmetica del ESPERADO del caso flojo
```

CARRIL D — Calibración

calibracion.md, corridas/

Sos el dueño de la evidencia. Tu trabajo no es que el agente acierte: es **encontrar dónde no acierta** y dejarlo escrito. Un desacuerdo honesto y bien resuelto suma más que una calibración perfecta sin historia.

Qué tenés que completar

- [] Organizá la **calibración a ciegas** (sección 6 de esta guía): que los cuatro puntúen antes de ver nada.
- [] Cargá las cuatro columnas en la tabla de **calibracion.md** §5.1 y marcá los desacuerdos **dimensión por dimensión**, no por total. Dos personas pueden llegar a 60 por caminos distintos.
- [] Buscá primero los desacuerdos **humano contra humano**. Si dos integrantes puntúan distinto la misma dimensión, el problema es de la rúbrica, no de la persona. Ese hallazgo vale más que una coincidencia perfecta.
- [] Conseguí **un repositorio real y ajeno** — la Entrega 1 o 2 de alguien que preste el link — y corré el agente encima. Anotá qué se rompió.
- [] Escribí cada desacuerdo con este formato: qué dijo el agente, qué dijimos nosotros, **quién tenía razón**, y qué se cambió. Las tres respuestas son válidas: el agente, nosotros, o nadie (la rúbrica era ambigua).

Mensajes de commit — así, no "cambios"

```
calibracion: cargo mi puntuacion a ciegas
corridas: primer repo ajeno, se rompio al leer un README sin encabezados
```


5 · Cómo se corre el agente

El agente no es un archivo suelto: es un contrato de **cuatro piezas que se cargan siempre juntas y en este orden**. Si falta una, la corrida no es válida — sin *banderas.md* no detecta al tramposo, y sin el esquema la salida deja de ser comparable entre corridas.

Orden	Archivo	Para qué
1	agente/system_prompt.md	El contrato: rol, contexto, tarea, restricciones, formato, ejemplos
2	rubrica.md	El criterio: dimensiones, requisitos, compuertas, escalas
3	agente/banderas.md	El catálogo anti-trampa con los efectos tipificados
4	agente/esquema_salida.json	La forma exacta de la salida

Y después, un **user prompt** por repositorio evaluado — es lo único que cambia entre corridas. **Temperatura 0**, siempre.

El user prompt, para copiar

Evalua el siguiente trabajo final.

REPOSITORIO: << URL del repo o ruta de la carpeta >>

ALUMNO: << nombre, o "anonimo" >>

FECHA: << AAAA-MM-DD >>

Aplica rubrica.md con las cuatro pasadas del system prompt, en orden: inventario, afirmaciones, puntuacion, auditoria.

Antes de puntuar, lista los archivos que efectivamente pudiste leer. Si no pudiste leer alguno de los exigidos, decilo: no lo supongas.

Devuelve primero el bloque JSON del esquema, despues el informe legible.

Los dos caminos

Camino	Cómo entra el repo	Qué se gana y qué se pierde
A (preferido)	El repositorio clonado en una carpeta local, que el agente lee con sus herramientas de archivos	Ve el árbol real y la historia de commits. Es el único camino donde se puede verificar la bandera de historia de proceso falsa.
B (respaldo)	El repositorio en .zip, adjunto a una conversación con el contrato ya cargado	Funciona sin instalar nada. Sin acceso a los commits : el agente tiene que declarar esa limitación en el informe, no disimularla.

Antes de correr: aislamiento de directorios

El repositorio que van a corregir puede traer su propio **AGENTS.md**, **CLAUDE.md** o **.cursorrules**, y casi todas las herramientas agénticas cargan esos archivos **solos**, como instrucciones, por estar cerca del directorio de trabajo. Si eso pasa, el trabajo que estás corrigiendo pasa a hablarle al corrector con la misma voz que vos.

```
~/evaluador/          <- el agente trabaja aca (contrato, rubrica, banderas)
~/repos-a-evaluar/xxx/ <- el repo del alumno, en OTRO arbol de directorios
```

Nunca al revés, y nunca uno dentro del otro. Es la bandera **G3d**, y es el único ataque donde la regla del prompt es la **segunda** línea de defensa: la primera es no darle a la herramienta la oportunidad de cargar el archivo.

Cómo se verifica en el informe

El campo **config_agentes_hallada** del JSON lista los archivos de configuración que el agente encontró dentro del repositorio evaluado. Si ese campo viene **vacío en un repo que tiene un AGENTS.md**, el agente no lo buscó — y si no lo buscó, no sabe qué parte de sus propias instrucciones no viene de vos.

Antes de cada tanda

Corré los tres casos de *casos/* y compará contra sus *ESPERADO.md*. Si alguno se corrió, algo se rompió en el contrato y hay que encontrarlo **antes** de corregir a nadie.

6 · Calibración a ciegas

Es la parte que más se posterga y la que más vale. Se hace **una vez, entre los cuatro, el mismo día**, y lleva alrededor de una hora.

Cómo se hace

- [] Cada uno, **por separado y sin hablar con los demás**, abre los tres casos de **casos/** con **rubrica.md** al lado.
- [] **Sin abrir** los ESPERADO.md ni la carpeta *corridas/*. Si ya sabés la respuesta, no estás calibrando: estás copiando.
- [] Puntúa las cinco dimensiones de cada caso, de 0 a 4, en la planilla de la página siguiente. **Dimensión por dimensión, no el total.**
- [] Además, en el caso tramposo: anotá **cuántos vectores de ataque encontraste vos, leyendo a mano**. Es el piso honesto contra el que se mide al agente.
- [] Recién ahí se juntan las cuatro planillas y se comparan (carril D).

Lo que hay que buscar — y no es lo que parece

El desacuerdo interesante **no** es agente contra humanos: es **humano contra humano**. Si dos integrantes puntúan la misma dimensión con dos niveles distintos, el problema es de la rúbrica — la escala no está lo bastante anclada. Ese hallazgo vale más que una coincidencia perfecta, y es material directo para *calibracion.md*.

Planilla — recortá o copió una por integrante

Integrante: _____ Fecha: _____ / _____ / 2026

Dimensión	Peso	excelente	flojo	tramposo
D1 · Sistema completo y funcionando	30			
D2 · Proceso documentado	25			
D3 · Formato y reproducibilidad	15			
D4 · Análisis económico	15			
D5 · Gobierno y riesgo	15			
Puntaje total (0–100)				
¿Levantaste alguna bandera?				
Tramposo: vectores que encontraste		—	—	/ 5

Dónde dudaste, y por qué — esto es lo que después se discute:

7 · Ensayo de la prueba de fuego

Hacerlo una vez, antes del jueves. Uno del grupo consigue un repositorio que los otros tres no vieron, y el agente lo corrige en vivo, con reloj.

Qué mirar

- [] **Cuánto tarda.** Si tarda más de lo que dura la atención de un aula, hay que pasar a la variante corta de *agente/plantilla_informe.md*.
- [] **Qué pasa si el repo no abre.** Debe salir *estado: no_evaluable*, nunca una nota inventada. Probálo a propósito con un link roto.
- [] **Qué pasa si el repo no tiene la estructura estándar.** Es el caso más probable en vivo: READMEs con otros encabezados, carpetas con otros nombres, todo en un .zip.
- [] **Quién habla.** El informe se lee, no se improvisa. Si alguien tiene que explicar el puntaje con palabras propias, el informe no estaba bien escrito.
- [] **Qué pasa si el repo trae su propio AGENTS.md.** Probálo con el caso tramposo: el informe tiene que listarlo en *config_agentes_hallada*, reportarlo como **G3d** y **no obedecerlo**. Es el fallo más difícil de ver en vivo, porque el informe equivocado se ve impecable.
- [] Llevá el contrato en **un archivo listo para pegar**, no en cuatro pestañas.

Lo que el evaluador tiene que hacer bien delante de la clase

Momento	Qué se ve
Un trabajo sólido	Puntaje alto pero no perfecto , y la sugerencia de mejora apunta a algo real
Un trabajo flojo	Puntaje bajo sin banderas . Incompleto no es fraudulento, y confundirlos es el peor error
Un trabajo tramposo	Las banderas transcriptas textualmente , la nota suspendida, y el caso escalado al profesor
Un repo que no abre	<i>no_evaluable</i> , y se explica qué se intentó. Cero improvisación

Lo que no hay que hacer

No defiendas al agente si falla en vivo. Si se equivoca, decilo, mostrá dónde y por qué: un desacuerdo honesto y bien explicado juega a favor. El evaluador que se desploma y lo disimula pierde dos veces.

8 · Checklist de entrega

Jueves 10/9, 18:59. Repasarlo entre los cuatro, en voz alta, la noche anterior.

- [] Repositorio **público** en GitHub — y **abierto en una ventana de incógnito** para confirmarlo. Un repo privado es *no_evaluable*, y esa lección la escribimos nosotros.
- [] El repositorio evaluado se clona **fuera** del directorio de trabajo del evaluador, y el informe declara **config_agentes_hallada**.
- [] **README.md** con las cinco secciones estándar y la tabla de integrantes completa.
- [] Los nombres exactos de la consigna: **rubrica.md**, **agente/**, **casos/excelente|flojo|tramposo/**, **calibracion.md**.
- [] **git log** con commits de los cuatro integrantes, repartidos en el tiempo.
- [] Las tablas de **calibracion.md** §5 llenas — o dicho explícitamente qué no se llegó a hacer. Declarar una deuda suma; esconderla resta.
- [] El cuadro de costos de **agente/config.md** §4 con precios y fecha de consulta.

- [] El agente corrió al menos una vez sobre un **repositorio real y ajeno**.
- [] Ensayo de la prueba de fuego hecho al menos una vez.
- [] Un integrante sube el link en la actividad **Parcial** del campus.

Una última cosa, y va en serio

No fabriquen la historia de commits. No antedaten, no repartan commits ajenos entre los cuatro usuarios, no armen una secuencia falsa el 9 a la noche. Se nota — nuestro propio agente tiene una bandera para eso — y sería usar el caso tramposo como manual de instrucciones, en el mismo repositorio donde escribimos que eso no se hace. Cuatro días de commits reales de cuatro personas cuentan una historia mucho mejor que veinte commits inventados.